

Algorithms Problem Solving

Level 3

Join String Overloading



Algorithms Problem Solving

Level 3

Join String Overloading

Definition

Function Overloading for String Joining

 Program provides multiple `JoinString()` functions that accept different container types.

 C++ selects the appropriate overload at compile time based on the arguments supplied.

Explanation

Vector Overload

-  The vector version receives a `vector<string>` and can determine the number of elements using `.size()`.

-  It is suitable for dynamic collections.

Array Overload

-  The array version receives a C-style `string[]` and an explicit length.

-  The function needs the length because a raw array parameter does not retain its original array-size information.

Compile-Time Resolution

-  Both functions can have the same name because their parameter lists differ.

-  The compiler selects the matching function signature during compilation.

Procedures

1. Implement `JoinString(vector<string>&, delim)`.
2. Implement `JoinString(string arr[], int length, delim)`.
3. Traverse the vector using `.size()`.
4. Traverse the array using the supplied `length`.
5. Join the elements using the delimiter.
6. Test both overloads.

💡 Example

Vector Join:

Sara, Alterkawi

➡ Result

Array Join:

Sara, Alterkawi

Note

- ◆ This is compile-time polymorphism, also known as static polymorphism.
- ◆ Function overloading depends on differences in parameter lists.
- ◆ Return type alone cannot be used to distinguish overloaded functions.

⊘ Warning

- ◆ A raw C-style array passed to a function normally decays to a pointer.
- ◆ `sizeof(arr)` inside the receiving function does not provide the original array length.
- ◆ Array boundaries must remain within the explicitly supplied `length`.



Comparison

Vector Overload

- ✓ Uses `vector<string>`.
- ✓ Determines the number of elements with `.size()`.
- ✓ Works naturally with dynamic STL containers.

Array Overload

- ✓ Uses a C-style `string[]`.
- ✓ Requires an explicit length.
- ✓ Supports older array-based interfaces.

✦ Summary

- ✓ Demonstrates function overloading in C++.
- ✓ Provides multiple implementations under the same function name.
- ✓ Supports both vectors and C-style arrays.
- ✓ Illustrates compile-time selection of overloaded functions.

Thank you 😊